Bob owns the document. This information is part of the entity data that the Cedar engine references during evaluation. This setup allows policies to rely on relationships such as ownership, membership, or management without needing those relationships included in every request.

If Bob tries to access the document from an unmanaged device, like his personal laptop, the scope would still match but the `is_managed` condition would fail. In that case, the policy would not apply, and unless another permit policy applied, the request would be denied.

This example shows how the request structure (PARC) and the policy structure (effect, scope, conditions) work together during evaluation. In the next section, we'll look more closely at Cedar's policy matching and decision semantics.

7.3.2 Evaluating policies

After receiving a request, the policy engine evaluates relevant policies to make a decision. In a medium- or large-scale organization like ACME Corp, policy logic is rarely located in a single place. Instead, it comes from multiple teams and systems, each with a specific purpose and scope. For example, ACME Corp probably has several policy stores, including the following:

- Security group policies enforce organization-wide security mandates, such as requiring managed devices or specific encryption standards.
- Application-specific policies are established by teams like Finance, HR, or Customer Collaboration to govern domain-level activities (for example, invoice approvals or document editing).

When a request comes in—such as Bob attempting to edit a document—the authorization layer gathers the relevant policies from these sources. Cedar treats them as a single policy set and evaluates them in one decision cycle.

HOW CEDAR MATCHES POLICIES TO REQUESTS

Cedar evaluates each request in two distinct stages: